Práctica de Clase — MLOps

Sistema: SalesPredict — Detección de Drift en Ventas

Repo: https://github.com/Miguel-Valles-uacj/mlops

Universidad Autónoma de Ciudad Juárez · Métricas del Software · 2026

Equipo 8:

- Gabriel Florentino Reyes - 223154
- Ian Luis Domínguez Ramírez - 222611
- Angel Eduardo Morales Hernández - 223287

Contexto

Eres parte del equipo MLOps de una cadena retail con presencia en 12 ciudades. Hace tres meses desplegaron **SalesPredict**, un modelo de clasificación que predice si una semana será de **ventas ALTAS o BAJAS**. El equipo comercial lo usa para planificar inventario y promociones.

En las últimas semanas el equipo comercial reporta predicciones incorrectas: el modelo clasifica semanas de temporada alta como normales. La **Accuracy bajó de 91% a 63%** y el **F1 cayó de 0.89 a 0.51**. Nadie tocó el modelo si no que algo cambió en los datos.

La práctica simula un equipo MLOps real que debe diagnosticar, documentar y reparar el pipeline antes de que el error impacte decisiones de negocio.

Flujo de Trabajo — Las 4 Fases

Cada equipo completa las cuatro fases en orden y documenta sus respuestas en este documento.

F1 **Detectar el problema**
Corre python run_pipeline.py, analiza las métricas y responde las preguntas guía.

F2 **Proponer una solución**
Sin ayuda externa: escribe tu hipótesis ANTES de tocar el código.

F3 **Implementar la solución**
Edita fix_pipeline.py. Puedes consultar documentación oficial de pandas, scipy y sklearn.

F4 **Evaluación**
Compara métricas antes vs después. Genera reporte_equipo.json. Responde preguntas de cierre.

Setup del Entorno

1. Clonar el repositorio

```
git clone https://github.com/Miguel-Valles-uacj/mlops
cd mlops/practica_mlops
git checkout equipo-N     # reemplaza N con tu número
git branch                          # debe aparecer: * equipo-N
```

2. Instalar dependencias

```
python -m pip install -r requirements.txt
```

3. Correr el pipeline base

```
python run_pipeline.py
# Verás las métricas en consola y la curva en reports/curva_degradacion.png
```

4. Al terminar, subir su trabajo

```
git add .
git commit -m "feat: fix pipeline equipo-N"
git push origin equipo-N
```

Datos del Sistema

Campo	Tipo	Descripción
`semana`	INTEGER	Número de semana del año
`tienda_id`	INTEGER	Identificador de sucursal
`categoria`	STRING	Categoría del producto
`precio_unitario`	FLOAT	Precio por unidad (puede estar en USD en producción)
`descuento_pct`	FLOAT	Porcentaje de descuento aplicado
`inventario_prev`	INTEGER	Inventario de la semana anterior
`es_temporada_alta`	BOOLEAN	1 si es temporada alta (Nov-Dic), 0 si no
`ventas_categoria`	STRING	ALTA o BAJA — variable objetivo (clasificación)

El modelo base es un ***Decision Tree (scikit-learn)***. Archivo: `models/modelo_base.pkl`

Métricas del Sistema

Son las 4 métricas que verán al correr el pipeline. Todas van de 0 a 1 (o 0% a 100%):

Métrica	Rango	Qué mide
`Accuracy`	0 – 1	De todas las semanas, ¿en qué porcentaje acertó el modelo?
`Precision`	0 – 1	Cuando dijo ALTA, ¿qué tan seguido tenía razón?
`Recall`	0 – 1	De todas las semanas realmente ALTAS, ¿cuántas detectó?
`F1`	0 – 1	Balance entre Precision y Recall. El más importante para este caso.

Ejemplo: Accuracy=0.63 significa que el modelo se equivoca en 37 de cada 100 semanas. F1=0.51 indica que falla especialmente en detectar semanas de ventas altas.

F1 — Detectar el Problema

Corre `python run_pipeline.py` y observa el reporte de métricas en consola. El script genera la curva de degradación en `reports/curva_degradacion.png`. Analiza la gráfica y responde:

Sobre las métricas:

- ***¿Qué métrica bajó más — Accuracy o F1? ¿En qué semana exacta empezó la caída según la curva?***
  La métrica que más falló fue F1, cayendo a 0. Hay varias caídas en las que se observan de F1. La primera caída comienza desde la semana 38 y termina en la semana 39. Empieza a crecer la gráfica y alcanza su pico en la semana 43, a partir de ahí, empieza a decrecer hasta llegar de nuevamente a 0 en la semana 45. Desde la semana 45 y siguientes se mantiene en 0.
- ***¿La caída fue gradual semana a semana o abrupta en un punto específico? ¿Qué implica cada caso sobre la causa del problema?***
  En la primera caída de F1, fue abrupta. En la segunda caída de F1 fue gradual. Las caídas primera caída abrupta fue a partir de la semana 38. La segunda caída abrupta fue desde la semana 44. Lo que implica en cada caso sobre la causa del problema apunta a un error en el pipeline de datos.
- ***¿El modelo falla más clasificando semanas ALTAS como BAJAS, o al revés? ¿Por qué ese tipo de error es más grave para el negocio?***
  Dado a que F1 cayó a 0, quiere decir que el recall está en 0, es decir, no está detectando las semanas ALTAS y las clasifica todas como BAJAS. Este error es grave para el negocio dado a que no se pueden tomar decisiones pertinentes para las semanas en donde se requieren de alta demanda, esto puede llevar a perdida de ventas.

Sobre los datos:

- ***Compara visualmente los datos de entrenamiento vs producción — ¿qué diferencias notas a simple vista en las distribuciones?***
  En los datos de entrenamiento, se marcan varias veces las semanas ALTAS, y las BAJAS como 0, sin embargo, esto no sucede con los datos de entrenamiento que no cuentan con ninguno número 1 en las columnas en semanas ALTAS, por lo tanto, no le importa este valor. Al igual que diferentes precios unitarios
- ***¿Hay alguna columna cuyos valores en producción se ven fuera de rango o en unidades distintas? ¿Cuál y por qué lo sospechas?***
  Los valores de alimentos, están en unidades USD. Analizando los datos, el precio menor es de 102 en producción. En los datos de entrenamiento el menor dato es 5.94 que demuestra que está utilizando otro tipo de cambio. Si se multiplica 5.94 * ~18 = 102.
- ***¿El problema afecta a todas las tiendas por igual o solo a algunas categorías o periodos? ¿Qué te dice eso sobre dónde buscar?***
  El problema es muy específico, es un problema que afecta a las categorías, particularmente en las semanas 45 a 52. Eso nos dice que hay un problema de cómo se están ingresando los datos en el pipeline.

Sobre el pipeline:

- ***¿En qué etapa del pipeline crees que entró el error — en los datos de entrada, en el preprocesamiento o en la inferencia? Justifica.***
  Como se menciona en los datos de entrada, Ya que, es posible que haya habido algún cambio de la moneda de los precios antes de entregarselos al pipeline. Debido a que afecta a todas las categorías a partir de una semana en específico, se infiere que el problema viene de fuera, de otra manera, el modelo estaría roto para todos los datos.
- ***¿Por qué el modelo no lanzó ningún error si los datos estaban mal? ¿Qué dice eso sobre cómo está construido el pipeline actual?***
  El modelo no puede distinguir qué datos están correctos o cuales no, eso es una lógica que es implementada por el desarrollador, el modelo solo consume los datos. Lo que dice eso sobre cómo está construido el pipeline actual es que carece de lógica para validar rangos de entrada

F2 — Proponer una Solución

Sin ayuda externa. Escribe tu propuesta ANTES de abrir fix_pipeline.py. Solo el equipo , sin IA

Hipótesis:

- ***¿Cuál es tu hipótesis de la causa raíz? Sé específico — no 'los datos cambiaron' sino qué columna, qué cambio exacto y por qué eso rompe la predicción.***

  El precio unitario está recibiendo el precio en USD desde la semana 45, lo cual está afectando a todas las categorías. El segundo problema es que en los datos de entrenamiento, no hay ningún valor 1 como en producción, lo que hace que el modelo aprenda a ignorar ese dato y no se registre en el modelo.
- ***¿Por qué el modelo funcionaba bien antes y ahora falla? ¿Qué asumió el modelo durante el entrenamiento que ya no se cumple en producción?***

  El modelo funcionaba porque asumió que siempre recibiría los precios unitarios en MXN, no en USD. Así, rompió la lógica con la cual se había implementado.
- ***Si hubiera dos causas posibles para el mismo síntoma, ¿cómo distinguirías cuál es la real antes de tocar el código?***

  Veríamos las gráficas para hacer una comparación entre las distribuciones, además, trataríamos de sacar los mínimos, máximos y el promedio de cada archivo de entrenamiento para ver irregularidades. Una causa natural es progresiva mientras que un bug, un error del pipeline se ve representado se ve abrupto y selectivo.

Plan de solución:

- ***¿Qué cambio específico proponen implementar en fix_pipeline.py? Descríbelo como pasos concretos, no como idea general.***

  1. Identificar qué filas de precio unitario están fuera de un rango que es el esperado, teniendo un mínimo establecido por los datos de producción.
  2. Convertir esos precios a MXN multiplicando por 17, valor del dólar,
  3. Verificar con los test que después de haber realizado las correcciones ya no difieren tanto.
  4. Pasar los datos corregidos al modelo para la generación de inferencias.
- ***¿Su fix ataca la causa raíz o solo los síntomas? ¿Cuál es la diferencia en este caso?***

  Ataca los síntomas, puesto que los datos que se obtienen, la fuente de estos, es la causa raíz y no hay manera de atacarlo. La diferencia es que, si el tipo de cambio cambia o se inserta alguna otra moneda, el fix fallaría.
- ***¿Qué métrica usarán para decidir si su fix funcionó — Accuracy, F1, Precision o Recall? ¿Por qué esa y no las otras?***

  F1 porque las semanas que son ALTA, las predice como BAJAS, es decir, tiene un F1=0. Además, no detectar una semana ALTA, es más grave que predecir ALTA cuando no lo es, esto podría hacer pérdida de ventas y tener producto demás. En producción el 53% de las semanas son ALTA, pero el modelo predice que todas son BAJA, es decir, un F1=0. Accuracy es una métrica engañosa, dado a que si el modelo dijera BAJA siempre, tuviera obtendría un porcentaje de 47% de las veces.
- ***¿Qué podría salir mal con su propuesta? ¿Qué efecto secundario podría tener en otras partes del pipeline?***

  Lo que vamos a hacer es codificar de forma fija el tipo de cambio, este es variable, puede cambiar a lo largo del tiempo, puede tanto subir como bajar, por lo que podría dejar desactualizado algunas filas en MXN. Además, puede ocurrir que algunas filas que estén en MXN, se multipliquen, lo cual es un error, se planea solo aplicar a las filas en donde el precio es menor al umbral establecido. El modelo nunca aprendió a usar la columna es_temporada_alta, por lo tanto, puede seguir fallando en detectar semanas de temporada alta.

F3 — Implementar la Solución

Edita `fix_pipeline.py`. El archivo tiene funciones vacías que debes completar. **No modifiques run_pipeline.py ni los archivos de datos.**

- **Restricción:** Usa únicamente las librerías de
- `requirements.txt`
- Implementa detect_drift(df_train, df_prod) usando el test KS de scipy — identifica qué columnas driftearon y con qué p-value
- Implementa el fix que propusiste en F2
- Tu código debe estar estructurado como funciones encadenadas, no como un script lineal
- Lanza excepciones descriptivas si los datos son inválidos — nunca silencies errores con try/except vacíos

Al terminar corre: `python run_pipeline.py --fixed` para ver las métricas con tu corrección aplicada y generar el reporte.

Durante la implementación responde:

- ***¿El test KS detectó drift en las columnas que sospechaban en F1? ¿Cuál fue el p-value de cada columna y qué significa ese número?***

  Detectó drift la columna que sospechábamos, en precio_unitario. Los valores fueron los siguientes:
  - precio_unitario p-value=0.0000
  - descuento_pct p-value=0.0000
  - inventario_prev p-value=0.0000
  - es_temporada_alta p-value = 1.0000

  El p-value signifca que la probabilidad que las dos distribuciones sean iguales, 0.0000 significa que no train y producción vengan de la misma distribución.
- ***¿Tuvieron que ajustar su hipótesis de F2 al ver los resultados del test KS? ¿Qué encontraron que no esperaban?***

  Las columnas que más sospechábamos eran las de precio_unitario y es_temporada_alta, pero solo tuvo drift precio_unitario y es_temporada_alta no tuvo drift, confirmando parcialmente la hipótesis. Tampoco se esperaba que descuento_pct e inventario_pct driftearan.
- ***¿Cómo estructuraron las funciones en fix_pipeline.py? ¿Por qué dividieron el código así y no de otra manera?***

  Se divideron las funciones de forma encadenada como se mencionan en las instrucciones de arriba. Creamos 3 funciones _detectar_precio_en_usd, encuentra si hay un error en los precios; _corregir_precio_usd, esta función aplica la conversión solo a las columnas afectadas. Y _corregir_temporada_alta normaliza la columna. Está organizado así para probar cada corrección independiente y tener claro cada parte.

F4 — Evaluar y Reflexionar

Corre `python run_pipeline.py --fixed`. El script genera automáticamente `reporte_equipo.json` con las métricas antes y después — solo llena tu nombre de equipo al inicio del archivo.

Evaluación de resultados:

- ***¿Cuánto mejoró cada métrica (Accuracy, Precision, Recall, F1)? Pon los números exactos de antes y después.***
  - Accuracy: original: 0.4956 | Con mejora: 0.6422 | + 0.1466
  - Precision: original: 0.8462 | Con mejora: 0.8713 | + 0.0251
  - Recall: original: 0.0466 | Con mejora: 0.3729 | + 0.3263
  - F1: original: 0.0884 | Con mejora: 0.5223 | + 0.4339
- ***¿Alguna métrica mejoró pero otra empeoró? ¿Qué trade-off estás aceptando y por qué vale la pena?***

  Todas las métricas mejorarón. No se está haciendo ningún trade-off.

- ***¿El modelo volvió al nivel original (Accuracy ~0.91, F1 ~0.89) o se quedó a la mitad? ¿A qué atribuyen la diferencia?***

  No. Se quedó a la mitad, subió a 0.64 y F1 a 0.52. Debido a que precio_unitario sigue con drift, la corrección de moneda no fue perfecta para todas las filas. El modelo sigue sin haber aprendido los patrones de comportamiento de es_temporada_alta. Para volver al nivel original es necesario reentrenar el modelo incluyendo datos de temporada alta.
- ***¿Confirmaron su hipótesis de F2 o la causa raíz resultó ser diferente? ¿Qué los sorprendió?***

  De forma parcial, se confirmó que los precios unitarios estaban siendo recibidos en USD y que es_temporada_alta nunca fue vista durante el entrenamiento. Lo que no se anticipó fue que descuento_pct e inventario_prev también tuvieron drift.

Razonamiento MLOps:

- ***¿En qué punto del pipeline falló primero y por qué ahí específicamente? Traza el camino del error desde los datos hasta la predicción incorrecta.***

  Al momento de consumir los datos. Se empezaron a guardar precios en USD, el CSV de producción se generó sin validación, el preprocesamiento los recibió y no realizó ninguna lógica, el modelo tomó los precios como MXN, interpretó registros como precios bajos, predijo BAJA que eran ALTA y F1 cayó a 0
- ***¿Cómo detectarías este problema automáticamente en producción, antes de que el equipo comercial se queje? ¿Qué alarma pondrías y en qué métrica?***

  Ejecutando la detección del drift cada semana antes de hacer una inferencia. La alarma se pondría en precio_unitario por categoría, si cae debajo de la mediana histórica, se activa la alarma. Y monitorear el recall en producción
- ***Define los umbrales de tu función check_model_health(): ¿cuándo es OK, WARNING y CRITICAL? Justifica cada número con base en las métricas que obtuviste.***

- OK: F1 >= 0.80 y Accuracy >= 0.85. Acercandóse al modela que opera cerca del nivel origina (F1=89, Accuracy=0.91)l. Aceptable para el negocio.
- WARNING: F1 >= 0.60 y Accuracy >= 0.70. Se coloca está por encima del nivel donde el equipo comercial empezó a quejarse (F1=0.51).
- CRITICAL: F1 < 0.60 o Accuracy < 0.70. Esto quiere decir que el modelo no está funcionando correctamente.

Decisión de negocio:

- ***¿Vale la pena reentrenar el modelo cada semana, cada mes o solo cuando se detecte drift? Justifica con los números que obtuviste — no con intuición.***

  Entre las semanas 38 y 44 el modelo operó con métricas razonables sin reentrenamiento. El colapso fue abrupto, a partide la semana 45. Reentrenar cada semana sería costoso. Sería detectar drift semanalmente y reentrenar cuando el KS supere el umbralde 0.05 en columnas críticas.
- ***Si tuvieras que explicarle el problema al equipo comercial (sin términos técnicos), ¿cómo lo describirías en dos oraciones?***

  El modelo aprendió usando precios en MXN, en noviembre se estuvieron recibiendo datos en USD. Haciendo que el sistema interprete productos caros como baratos y predijera semanas normales cuando eran de alta demanda.
- ***¿Qué cambiarías en el pipeline original para que este tipo de falla no vuelva a ocurrir silenciosamente?***

  Validación de los datos fuente, si precio_unitario cae debajo del umbral histórico, rechazar el lote y lanzar alerta antes de guardar el csv. Y correr el drift automáticamente cada semana y bloquear cualquier inferencia si algunas columna crítica supera el umbral KS.

Entregables por Equipo

`fix_pipeline.py`	Su solución implementada con funciones estructuradas

`reporte_equipo.json`	Generado automáticamente al correr run_pipeline.py --fixed
`Push a rama`	git push origin equipo-N con ambos archivos
`Visualizacion en Power Bi`	

Visualizacion en Power BI

Al terminar la Fase 4, usen los datos generados por el pipeline para crear un reporte visual en Power BI Desktop. El pipeline ya genera los archivos csv en reports/ con todos los datos listos para importar.

Pasos:

- Abrir **Power BI Desktop** (si no lo tienen instalado, descargarlo desde microsoft.com)
- Ir a **Inicio** → **Obtener datos** → **Texto/CSV** y seleccionar los archivos csv de su carpeta del proyecto. Hacer clic en **Cargar**.
    - **Gráfica de líneas — Curva de degradación**
    - Seleccionar gráfico de líneas
    - Usar la columna semana como eje X
    - Usar accuracy_base y accuracy_fix como valores

      Crear la segunda visualización obligatoria:
      Gráfica de barras — Métricas antes vs después
    - Seleccionar gráfico de barras agrupadas
    - Usar la columna metrica como eje X
    - Usar baseline y fix como valores

  Crear la segunda visualizacion obligatoria: **Grafica de barras — Metricas antes vs despues**.
- Agregar una **tarjeta (Card)** con el nombre del equipo. En el panel de visualizaciones seleccionar **Tarjeta** y arrastrar el campo `equipo` del CSV.
- Guardar el reporte como `reporte_equipo.pbix` dentro de la carpeta del proyecto usando **Archivo** → **Guardar como** y nombrarlo exactamente `reporte_equipo.pbix` para que el git add . lo incluya automaticamente al hacer push.

Nota: Power BI Desktop es para windows

Reglas Generales

- Trabajar únicamente en fix_pipeline.py. No modificar run_pipeline.py ni los archivos de datos.
- En F1 y F2 no está permitido el uso de IA generativa (ChatGPT, Copilot, Claude, Gemini, etc.). En F3 pueden consultar únicamente documentación oficial de scikit-learn, pandas y scipy.
- Si una función recibe datos inválidos, lanzar excepciones descriptivas. No silenciar errores con try/except vacíos.
- Las respuestas a las preguntas van escritas directamente en este documento, en la sección correspondiente a cada fase.
- Commit al terminar cada función antes de continuar a la siguiente fase.